🚀 Collections Framework Masterclass

Chapter 18 — WeakHashMap — Complete Deep Dive

> **Interview Importance:** ⭐⭐⭐⭐☆

`WeakHashMap` is one of those Java collections that interviewers use to distinguish between someone who **knows the API** and someone who understands **Java references + Garbage Collection**.

The central idea:

> **WeakHashMap allows entries to disappear automatically when their keys are no longer strongly reachable elsewhere.**

The important word is **key**.

```
WeakHashMap
     ↓
Weak references to keys
     ↓
GC can collect unreachable keys
     ↓
Corresponding mappings disappear
```

---

1. What Is WeakHashMap?

Package:

```java id="m5w0ue"
java.util.WeakHashMap
```

```
It implements:

```text id="5z0l7u"
Map<K,V>
```

Basic example:
```

```java id="r7k2v9" Map<Key, String> map = new WeakHashMap<>();

```
Unlike `HashMap`, the Map does not strongly retain its keys.

---

2. HashMap vs WeakHashMap

This is the most important distinction.

HashMap

```text id="j1z2k6"
HashMap
   ↓
strong reference
   ↓
Key
```

As long as the HashMap contains the key, the key remains strongly reachable through the Map.

WeakHashMap

```text id="d9p3a4" WeakHashMap ↓ weak reference ↓ Key

```
If there are no other strong references to the key, the key can become eligible
for garbage collection.

---

3. What Is a Strong Reference?

Most normal Java references are strong references.

```java id="x4f7j2"
Employee employee =
    new Employee();
```

Here:

```text id="c3m5y8" employee ↓ Employee object
```

```
As long as `employee` is strongly reachable, the object isn't eligible for GC
merely because no other reference exists.

---

# 4. What Is a Weak Reference?

A weak reference does not prevent an object from being garbage collected.

Conceptually:

```text id="n6s2r8"
Strong reference:

variable ————————→ Object
                     ↑
                 reachable


Weak reference:

WeakRef ——————————→ Object
                      ↑
              doesn't keep alive
```

This is the behavior WeakHashMap relies on.

---

5. The Key Relationship

Consider:

```java id="y8v1k3"
Key key = new Key();

WeakHashMap<Key, String> map = new WeakHashMap<>();

map.put(key, "Data");
```

```
There are now two conceptual references:

```text id="v4h7q1"
key variable ————————→ Key
```

```
                          ↑
WeakHashMap ──weak────────┘
```

The key is still strongly reachable because:

```text id="n9x2c5" key variable
exists.

---

# 6. What Happens When the Strong Reference Disappears?

Now:

```java id="e6s3w0"
key = null;
```

Conceptually:

```text id="p2y7v5" key variable ↓ null

WeakHashMap | weak ref ↓ Key

If there are no other strong references to that Key:

```text id="z1c9m4"
Key becomes eligible for GC
```

After garbage collection and WeakHashMap's cleanup processing, the corresponding entry can disappear.

---

7. Very Important: GC Is Not Deterministic

Don't say:

> "Setting key = null immediately removes the entry."

That's incorrect.

The correct sequence is:

```text id="q8r4s2" Strong reference disappears ↓ Key becomes eligible for GC ↓ GC may collect key ↓ Reference processing occurs ↓ WeakHashMap expunges stale entry

```
Garbage collection timing is not deterministic.

---

8. Example

```java id="u3f7k9"
WeakHashMap<Object, String> map =
    new WeakHashMap<>();

Object key = new Object();

map.put(key, "Hello");

System.out.println(map.size());
```

Output:

```text id="p6y2w4" 1

```
Now:

```java id="h1q8v6"
key = null;
```

The entry becomes eligible for eventual removal once the key is collected.

You should **not** write code that assumes the next statement will definitely produce:

```text id="v8k2m5" 0

```
---

9. Why Doesn't the Value Keep the Key Alive?

This is a subtle but important point.

Conceptually, WeakHashMap is designed so that the map's internal entry does not create a strong path from the map to its key.
```
```

```
However, you must be careful if the value itself strongly references its key.

Example:

```java id="n2c7x1"
Key key = new Key();

map.put(key, key);
```

Now:

```text id="f8k4q6" WeakHashMap ↓ Entry ↓ Value ——————→ Key
```

```
The value itself can provide a strong path to the key.

Therefore the key may not become collectible as expected.

---

# 10. The Key Must Be Weakly Reachable

The basic condition is:

```text id="k3p7m2"
No strong reference path to key
          ↓
Key eligible for GC
```

The WeakHashMap's weak reference alone does not keep it alive.

---

11. WeakHashMap Uses Weak References Internally

The implementation uses mechanisms from:

```java id="d5w2n8" java.lang.ref
```

```
particularly weak references.

Conceptually:
```

```text id="u8m3r1"
WeakHashMap
     ↓
Entry
     ↓
WeakReference<Key>
```

The implementation can then detect when the referenced key has been cleared.

---

12. ReferenceQueue

A key implementation concept is:

```java id="y6c4t9" ReferenceQueue
```

```
A simplified lifecycle:

```text id="j7v2n6"
Key
 ↓
WeakReference
 ↓
Key becomes unreachable
 ↓
GC clears weak reference
 ↓
Reference enqueued
 ↓
WeakHashMap processes queue
 ↓
Entry removed
```

This is a very good senior-level interview answer.

---

13. Why Does WeakHashMap Need ReferenceQueue?

Suppose:
```

```text id="a4m7q2"
Key gets garbage collected
```

```
The Map must eventually remove the corresponding entry.

The `ReferenceQueue` provides a mechanism for detecting processed weak
references.

Conceptually:

```text id="e1n8p5"
Garbage Collector
       ↓
clears weak reference
       ↓
ReferenceQueue
       ↓
WeakHashMap cleanup
       ↓
remove stale entry
```

---

14. Does WeakHashMap Run a Background Thread?

No.

This is an important misconception.

WeakHashMap doesn't require a dedicated background thread constantly scanning all keys.

Cleanup is integrated with Map operations and reference processing.

In simplified terms:

```text id="z6r4p8"
Map operation ↓ process stale references ↓ remove obsolete entries
```

```
The exact implementation details can vary by Java version.

---

# 15. `size()` Can Change Unexpectedly
```
```

```
This is one of the most important WeakHashMap interview traps.

Suppose:

```java id="b8n3k1"
WeakHashMap<Key, String> map =
    new WeakHashMap<>();
```

You insert several keys.

Then external strong references disappear.

After GC:

```text id="r9p5x2" map.size()
```
can become smaller.

Therefore:

> **The contents of WeakHashMap can change as a side effect of garbage collection.**

This is fundamentally different from normal Map implementations.

---

16. Why This Matters

With HashMap:

```java id="j5q7r2"
map.put(key, value);
```
```

the entry normally remains until you explicitly remove it or clear the Map.

With WeakHashMap:

```text id="n6k1v4" key becomes unreachable ↓ GC ↓ entry may disappear
```
```
So the Map's contents are partly dependent on object reachability.

---
```

```
# 17. WeakHashMap Is Not a Normal Cache

This is a common interview misconception.

You might hear:

> "WeakHashMap is an LRU cache."

**No.**

LRU means:

```text id="s5h2p9"
Least Recently Used
```

which is about access recency.

WeakHashMap is about:

```text id="e7j3k1" Object reachability

```
They solve different problems.

---

18. WeakHashMap vs LRU Cache

WeakHashMap

```text id="f6x4w2"
Key reachability
      ↓
GC
      ↓
Entry disappears
```

LinkedHashMap LRU

```text id="k8r2m5" Access order ↓ Least recently used ↓ Evict entry

```
Therefore:
```

```text id="n5c9q7"
WeakHashMap ≠ LRU Cache
```

---

19. Typical Use Case

WeakHashMap is useful when you want:

> "Associate metadata with an object without making the object stay alive solely because of that association."

Example concept:

```text id="p3h7m1" Object ↓ metadata

```
You don't want:

```text id="v6q4z8"
metadata map
   ↓
Object kept alive forever
```
```

WeakHashMap can help avoid that retention.

---

# 20. Example — Object Metadata

Suppose you need metadata for objects:

```java id="x8f2n5" WeakHashMap<Object, String> metadata = new WeakHashMap<>();

Object object = new Object();

metadata.put(object, "Temporary metadata");
```
As long as:

```text id="c5m9r3"
object
```
```

is strongly reachable, the mapping can exist.

Once the object is otherwise unreachable, the mapping can eventually disappear.

---

21. Example — Listener/Metadata Associations

Weak references can be useful in scenarios involving:

```text id="q2k7m4"
metadata
temporary associations
object lifecycle tracking
```

```
where you don't want the tracking structure itself to extend the lifetime of the
tracked object.

The exact design should still be evaluated carefully; WeakHashMap is not
automatically the right solution for every memory-management problem.

---

# 22. Null Key

WeakHashMap allows:

```java id="m4p8s2"
map.put(null, "Hello");
```
```

Unlike ordinary weakly referenced keys, `null` itself cannot be garbage collected.

So the null-key mapping does not disappear merely because of weak-key processing.

---

23. Null Values

WeakHashMap also allows null values:

```java id="r7x3k5"
map.put(key, null);
```

```
There is no special restriction comparable to `ConcurrentHashMap`.

---

# 24. WeakHashMap vs HashMap
```

```
| Feature | HashMap | WeakHashMap |
|---|---|---|
| Key reference | Strong | Weak |
| Entries disappear due to GC | ❌ | ✅ possible |
| Ordering | None guaranteed | None guaranteed |
| Null key | One | One |
| Null values | Yes | Yes |
| Thread-safe | ❌ | ❌ |
| Normal general-purpose Map | ✅ | ❌ |
| Object-lifecycle-sensitive use cases | Limited | ✅ |

---

# 25. WeakHashMap vs LinkedHashMap

| Feature | WeakHashMap | LinkedHashMap |
|---|---|---|
| Primary purpose | Weak-key associations | Predictable ordering |
| Ordering | None guaranteed | Insertion/access |
| Automatic GC-based removal | ✅ | ❌ |
| LRU support | ❌ | ✅ |
| Thread-safe | ❌ | ❌ |

---

# 26. WeakHashMap vs ConcurrentHashMap

| Feature | WeakHashMap | ConcurrentHashMap |
|---|---|---|
| Weak keys | ✅ | ❌ |
| Thread-safe | ❌ | ✅ |
| Null key | One | ❌ |
| Null value | ✅ | ❌ |
| Main purpose | Weak associations | Concurrent access |

Don't choose WeakHashMap just because you want memory optimization in a
multithreaded application.

Concurrency requirements are separate.

---

# 27. Strong Reference Path — Important

Suppose:
```

```java id="z3n8v5"
Object key = new Object();

Map<Object, String> map =
    new WeakHashMap<>();

map.put(key, "Data");
```

There is:

```text id="s8f2w6" key variable ↓ Object
```

```
Therefore the object is still strongly reachable.

The weak map entry does not cause collection while another strong reference
exists.

---

# 28. After `key = null`

```java id="n6r4p1"
key = null;
```

If no other strong references exist:

```text id="a2v7k9" WeakHashMap ↓ weak reference ↓ Object
```

```
Now the object is eligible for GC.

Eventually:

```text id="j5m8x3"
Object collected
      ↓
weak reference cleared
      ↓
entry removed
```

---

29. `System.gc()` Trap

You might see examples like:

```java id="e7x2p6" key = null; System.gc();

```
and expect:

```java id="q3m9w1"
map.size() == 0
```

Don't rely on this.

`System.gc()` is only a request/hint to the JVM.

It doesn't guarantee immediate garbage collection.

Therefore:

> WeakHashMap behavior must never depend on a specific GC timing.

---

30. WeakHashMap Is Not About Weak Values

This is extremely important.

WeakHashMap's special behavior is primarily:

```text id="f4p7s2" WEAK KEYS

```
not:

```text id="w9k3n6"
WEAK VALUES
```

The values are not automatically weak simply because the Map is a WeakHashMap.

---

31. Key vs Value

Think:

```text id="d2m8v5" WeakHashMap<K,V>

K → weakly referenced V → ordinary reference

```
Therefore:

```text id="h7q4x9"
Key lifecycle
    ↓
GC-sensitive

Value lifecycle
    ↓
not automatically weak
```

---

32. Value Can Accidentally Keep Key Alive

This is a sophisticated interview question.

Suppose:

```java id="r4k7p2" class Data { Key key; }

```
Then:

```java id="v9x2m6"
Key key = new Key();
Data data = new Data(key);

map.put(key, data);
```

Now:

```text id="g5n8c3" WeakHashMap ↓ value ↓ Data ↓ Key
```

```
The value strongly references the key.

Therefore the key may remain reachable.

This defeats the intended weak-key behavior.

---

# 33. General Rule

When using WeakHashMap:

> **Ensure the value does not unintentionally maintain a strong reference back
to the key.**

This is particularly important in metadata/cache designs.

---

# 34. WeakHashMap and Garbage Collection

The lifecycle is:

```text id="h6r3n9"
         Key exists
              ↓
      Strongly reachable
              ↓
        Map contains entry
              ↓
 Strong references disappear
              ↓
       Key eligible for GC
              ↓
          GC runs
              ↓
    Weak reference cleared
              ↓
      Reference queued
              ↓
 WeakHashMap expunges entry
```

This is the most important diagram in this chapter.

```

35. Does WeakHashMap Guarantee Immediate Removal?

No.

The correct wording is:

> **Entries whose keys have been garbage collected become eligible for removal, and the Map removes stale entries as it processes cleared references.**

Don't say:

> "The entry disappears immediately when the key becomes unreachable."

---

36. Does WeakHashMap Prevent Memory Leaks?

It can help prevent certain forms of memory retention.

But:

> **WeakHashMap is not a universal memory-leak solution.**

You still need to understand:

- Strong references
- Object graphs
- Thread-local references
- Static references
- Listener registrations
- Caches
- Resource lifecycle

Weak references are one tool, not a magic solution.

---

37. Iteration Behavior

Because entries can disappear due to GC, iteration should not assume that a previously observed entry will remain indefinitely if its key has become weakly reachable.

This is another consequence of GC-sensitive Map contents.

38. `entrySet()`

Like other Maps:

```java id="k5z8x2" map.entrySet();
```

```
provides the Map's entries.

You can iterate:

```java id="p8v3m6"
for (Map.Entry<Key, String> entry
        : map.entrySet()) {

    System.out.println(
        entry.getValue()
    );
}
```

But remember:

> The Map is GC-sensitive.

---

39. Is WeakHashMap Ordered?

No ordering guarantee.

If you need insertion order:

```text id="x7n4q2" LinkedHashMap
```

```
If you need sorted keys:

```text id="m3v8p6"
TreeMap
```

WeakHashMap's purpose is weak-key lifecycle behavior.

---

40. Is WeakHashMap Thread-Safe?

No.

If multiple threads access it concurrently and at least one structurally modifies it, you need an appropriate synchronization strategy.

There isn't a standard `ConcurrentWeakHashMap` in the Java Collections Framework.

---

41. WeakHashMap and Synchronization

You can wrap a Map using:

```java id="b6x9r4" Collections.synchronizedMap( new WeakHashMap<>() );

```
But this does not magically create every property of a specialized concurrent collection.

Iteration still requires following the normal synchronization rules for synchronized collection wrappers.

---

42. WeakHashMap vs `WeakReference`

Don't confuse the two.

WeakReference

A reference mechanism:

```java id="q5m7n3"
WeakReference<T>
```

WeakHashMap

A Map implementation that uses weak references for keys.

Conceptually:

```text id="j9p2x6" WeakReference ↓ building block
```

WeakHashMap ↓ collection abstraction using weak keys

```
---

# 43. Reference Types in Java

You should know these four categories:

```text id="u6r3p9"
Strong
Soft
Weak
Phantom
```

For this chapter, the important one is:

```text id="v8k2m4" WeakReference
```

```
A weak reference does not prevent collection when the referent is otherwise
strongly unreachable.

---

# 44. Weak vs Soft Reference

Interviewers sometimes ask:

> WeakHashMap uses weak references. Why not soft references?

Conceptually:

### Weak

Object can be collected once it is only weakly reachable.

### Soft

Historically intended for objects that could be retained until memory pressure,
although exact GC behavior is JVM-dependent.

WeakHashMap specifically uses weak references because its goal is **lifecycle
association**, not a memory-pressure cache policy.

---
```
```

```
# 45. WeakHashMap and Cache Design

A common mistake:

> "I'll use WeakHashMap as my application cache."

Be careful.

A cache generally requires explicit semantics around:

```text id="d3k7m1"
size
eviction
expiration
refresh
concurrency
hit rate
memory limits
```

WeakHashMap instead says:

```text id="h8n4q5" If nobody strongly needs this key, the association can disappear.

```
Those are very different semantics.

---

46. Example — Temporary Object Metadata

Consider:

```java id="m6x2c8"
class Parser {
    ...
}
```

You create metadata associated with parser-created objects:

```java id="k9v4r2" WeakHashMap<Object, Metadata> metadata = new WeakHashMap<>();

```
The metadata association doesn't necessarily need to extend the lifetime of the
object.

This is a much better conceptual use case than treating WeakHashMap as a
```

````
traditional cache.

---

# 47. ⭐ Interview Question

### What makes WeakHashMap different from HashMap?

Strong answer:

> HashMap strongly references its keys, whereas WeakHashMap uses weak references
for keys. If a WeakHashMap key is no longer strongly reachable elsewhere, it can
be garbage collected and the corresponding mapping can subsequently be removed.

---

# 48. ⭐ Interview Question

### When does an entry disappear from WeakHashMap?

Don't say:

> "When the key is set to null."

Say:

> "When the key is no longer strongly reachable, it becomes eligible for garbage
collection. After the key is collected and the weak reference is processed,
WeakHashMap can remove the corresponding stale entry."

---

# 49. ⭐ Interview Question

### Does setting the key variable to null immediately remove the entry?

**No.**

```text id="j3m7p9"
key = null
````

only removes one strong reference.

GC and reference processing determine when the key can actually be collected and the mapping cleaned up.

````

50. ⭐ Interview Question

Does WeakHashMap use weak values?

No.

Its special semantics concern:

```text id="w2k6r8" keys

```
The values remain normally referenced by the entries.

---

51. ⭐ Interview Question

Can the value prevent the key from being garbage collected?

Yes.

If the value strongly references the key, a strong path can exist:

```text id="c4p8m2"
WeakHashMap
    ↓
  value
    ↓
   key
```

Therefore the key may remain strongly reachable.

---

52. ⭐ Interview Question

Is WeakHashMap thread-safe?

No.

It requires external synchronization or another concurrency design when used concurrently.

---

53. ⭐ Interview Question

Is WeakHashMap an LRU cache?

No.

LRU is based on access recency.

WeakHashMap is based on key reachability and garbage collection.

---

54. ⭐ Interview Question

Why use WeakHashMap?

Strong answer:

> When you need a mapping associated with an object's lifecycle but don't want the Map itself to keep the key alive. It is useful for metadata or temporary associations where entries should naturally disappear when their keys become otherwise unreachable.

---

55. ⭐ Coding Question

What happens?

```java id="h2m7q4" WeakHashMap<Object, String> map = new WeakHashMap<>();

Object key = new Object();

map.put(key, "Hello");

key = null;
```

```
Can the entry eventually disappear?

**Yes.**

If no other strong reference to the key exists, the key becomes eligible for GC,
and after collection/reference processing the mapping can be removed.

---
```

```
# 56. ⭐ Coding Question

What is guaranteed here?

```java id="u8r2m6"
Object key = new Object();

map.put(key, "Hello");

key = null;

System.out.println(map.size());
```

Answer:

Nothing deterministic about whether the entry has already disappeared.

The key may not have been collected yet.

Therefore you cannot reliably predict the size from this code alone.

---

57. ⭐ Coding Question

Consider:

```java id="c7p4m1" Object key = new Object();

map.put(key, key);

key = null;

```
Will the entry necessarily disappear?

No.

The value may strongly reference the same key.

Conceptually:

```text id="x9m5q2"
Map
 ↓
Value
```
```

```
  ↓
Key
```

That strong path can prevent collection.

---

58. ⭐ Coding Question

Which Map should you choose?

Requirement:

> Associate metadata with an object without making the metadata map keep that object alive.

Answer:

```java id="z4k8n6" WeakHashMap<Object, Metadata>
```

```
---

# 59. WeakHashMap Decision Framework

```text id="j5r7m2"
Need key-value association
          ↓
Do keys need normal strong lifetime?
          ↓
      YES ————→ HashMap
          |
         NO
          ↓
Should entries disappear when
keys become otherwise unreachable?
          ↓
        YES
          ↓
     WeakHashMap
```

Ordering requirement:

```text id="s8n2q5" Insertion/access order ↓ LinkedHashMap
```
```

````
Sorted keys:

```text id="w6p3k9"
TreeMap
````

Concurrency:

```text id="m4r7x1" ConcurrentHashMap

````
---

60. HashMap → LinkedHashMap → TreeMap → WeakHashMap

At this point, your Map hierarchy should mentally look like:

```text id="d7m3v8"
                        Map
                         |
        ┌────────────────┼────────────────┐
        ↓                ↓                ↓
    HashMap        LinkedHashMap        TreeMap
        ↓                ↓                ↓
  Fast lookup      Ordering          Sorted keys
                         |
                   Access order
                         ↓
                    LRU-style
````

And separately:

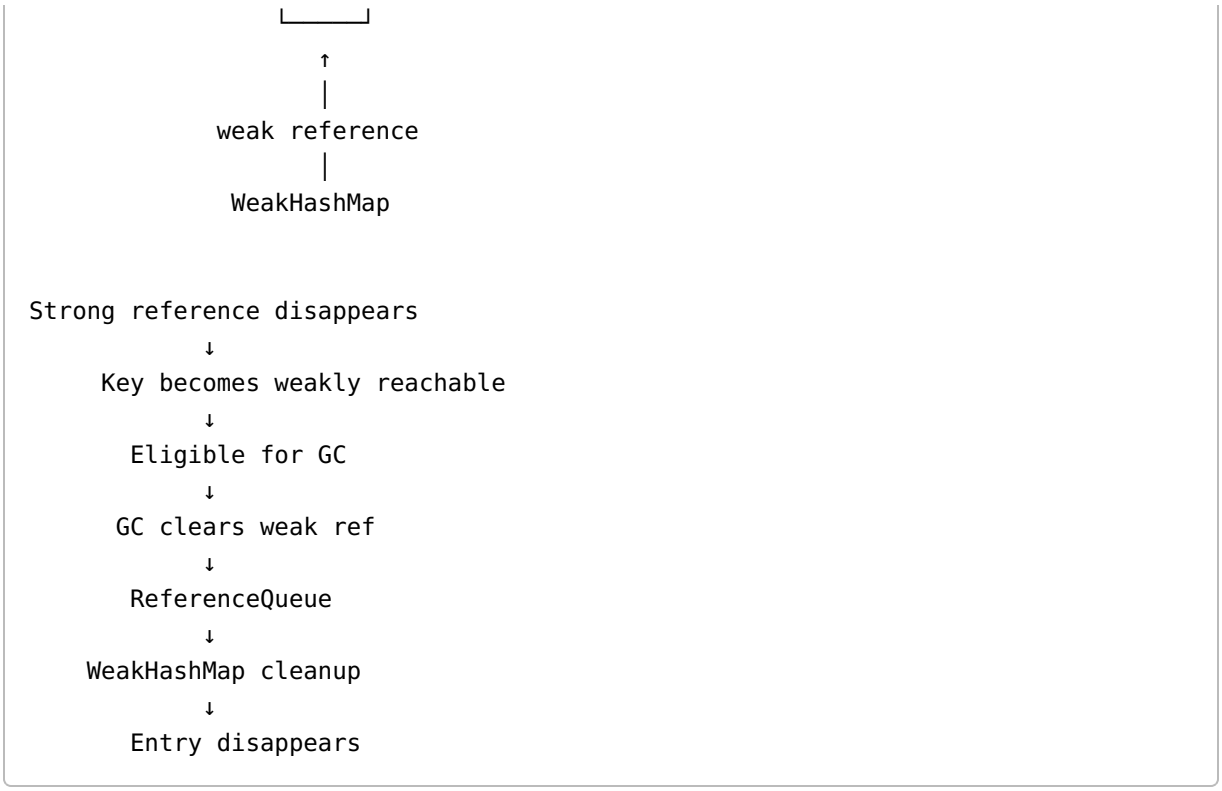
```text id="q2n8c5" WeakHashMap ↓ Weak keys ↓ GC-sensitive lifecycle

````
---

61. 🔥 Most Important WeakHashMap Diagram

Memorize this:

```text id="r8m4q2"
            Strong reference
                   ↓
               ┌─────┐
               | Key |
````
````



That's the entire concept.

---

62. 🔥 Interview One-Liner

If the interviewer asks:

> **"Explain WeakHashMap in one minute."**

Answer:

> "`WeakHashMap` is a Map implementation whose keys are held using weak references. Unlike HashMap, the Map itself doesn't strongly keep its keys alive. If a key is no longer strongly reachable elsewhere, it can be garbage collected, after which the corresponding mapping can be removed. This makes WeakHashMap useful for lifecycle-sensitive metadata or temporary associations. Its behavior is GC-dependent, it isn't thread-safe, and it shouldn't be confused with an LRU cache."

That's a strong senior-level answer.

🧠 Final Mental Model

```text id="n5k8r3"
HashMap ↓ Strong keys ↓ Map helps keep key alive

WeakHashMap ↓ Weak keys ↓ Map doesn't keep key alive ↓ Key becomes unreachable ↓ GC ↓ Entry can disappear
```

```
Remember:

```text id="w3p7m1"
WeakHashMap
      ↓
WEAK KEYS
      ↓
GC-SENSITIVE
```

Not:

```text id="k8q4n6"
Weak values
```

```
Not:

```text id="v2m9r5"
LRU
```

Not:

`text id="c6x1p8"`
`Thread-safe`

🔥 Chapter Summary

Concept	Key Point
WeakHashMap	Map with weakly referenced keys
Package	`java.util`
Main purpose	Lifecycle-sensitive associations
Key reference	Weak

Concept	Key Point
Value reference	Normal/strong by default
GC-sensitive	✅
Automatic stale-entry cleanup	✅
Immediate removal	❌ Not guaranteed
ReferenceQueue	Used for stale-reference processing
Null key	Allowed
Null values	Allowed
Ordering	None guaranteed
Thread-safe	❌
LRU cache	❌
Strong value → key reference	Can keep key alive
Common use	Object metadata / temporary associations

🎯 Interview Readiness Checklist

You should be able to explain:

- [ ] What WeakHashMap is
- [ ] HashMap vs WeakHashMap
- [ ] Strong vs weak references
- [ ] Why weak keys can be garbage collected
- [ ] What happens after `key = null`
- [ ] Why GC timing is nondeterministic
- [ ] `WeakReference`
- [ ] `ReferenceQueue`
- [ ] Stale-entry cleanup
- [ ] Why `size()` can change
- [ ] Why WeakHashMap isn't an LRU cache
- [ ] Why WeakHashMap isn't thread-safe
- [ ] Weak keys vs weak values
- [ ] How a value can accidentally keep a key alive
- [ ] Appropriate real-world use cases
- [ ] WeakHashMap vs HashMap
- [ ] WeakHashMap vs LinkedHashMap
- [ ] WeakHashMap vs ConcurrentHashMap

🚀 Next Chapter

Chapter 19 — IdentityHashMap (Complete Deep Dive)

We'll focus on what's new:

- `==` vs `equals()`
- Identity-based key comparison
- `System.identityHashCode()`
- How IdentityHashMap differs internally from HashMap
- Why it uses reference identity
- Duplicate-looking keys that coexist
- IdentityHashMap's unusual `Map` behavior
- When identity semantics are useful
- Serialization/object graph use cases
- IdentityHashMap vs HashMap
- IdentityHashMap vs `WeakHashMap`
- Tricky `equals()` / `hashCode()` interview questions
- 20+ interview questions